

Calgary Residential Development ROI Classifier

CMPT 310 - D100 Introduction to Artificial Intelligence, Summer 2026

Finley Thomson · Monem Moheq · Srinivas Suggu · Jarrod Hwang

224

completed Calgary projects

7

pre-outcome inputs

3

ordered ROI classes

5

stratified evaluation folds

1. System explanation

We built an offline supervised-learning pipeline that classifies completed Calgary residential developments as **Low, Medium, or High ROI**. Its intended use is research and early-stage triage: ranking projects for deeper expert review. It is not a deployed valuation service and must not be used as financial advice.

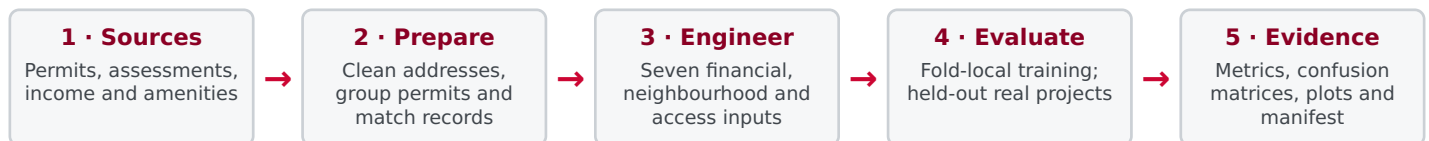
Input, target, and output

The final modeling table combines building-permit and historical assessment records with community-income and proximity features. Application years span 2017–2023 and completion years span 2018–2023. The classes are nearly balanced: 75 Low, 74 Medium, and 75 High.

$$\text{ROI} = [\text{Final Assessment} - (\text{Initial Assessment} + \text{Estimated Development Cost})] \div (\text{Initial Assessment} + \text{Estimated Development Cost})$$

The classifier receives seven values available before the outcome: initial assessment, community income, cost per unit, distance to downtown, and distance to the nearest transit stop, park, and public school. Final assessment, ROI, and Classification are explicitly excluded from prediction features.

AI and evaluation pipeline



The canonical evaluator compares a majority-class baseline, scikit-learn random forest, and optional custom random forest. A separate experiment generates 3,000 multivariate-lognormal rows from each fold's training data. Synthetic rows never enter the held-out evidence.

Reliability controls: fixed seed 310; stratified folds; train-fold-only synthetic generation; outcome-column and schema checks; an explicit baseline; accuracy, macro-F1, balanced accuracy, ordinal MAE, and severe Low↔High error reporting.

Evaluation, results, and limitations

Each real project appears once in held-out evidence, producing 224 out-of-fold predictions. Forests use 100 trees, maximum depth 7, three candidate features per split, and minimum leaf size 5. Macro-F1 gives each class equal weight; ordinal MAE measures how far predictions move across the ordered labels.

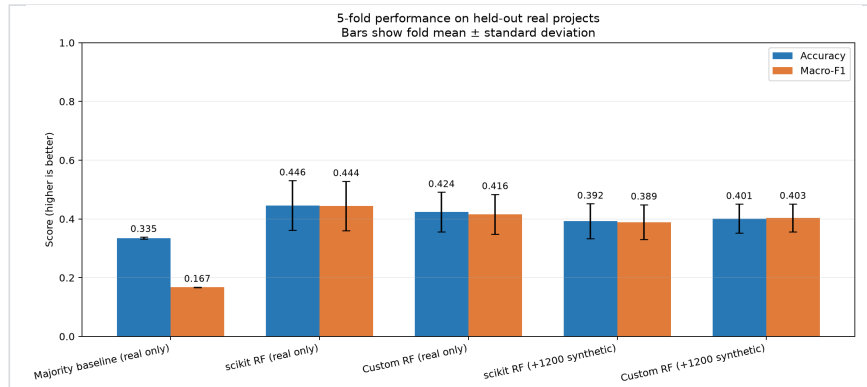


Figure 1. Five-fold mean $\pm$ standard deviation on held-out real projects.

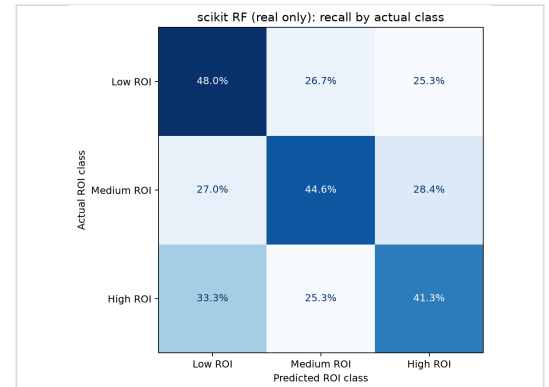


Figure 2. Row-normalized confusion matrix; diagonal cells are class recall.

Model	Training data	Accuracy	Macro-F1	Ordinal MAE	Severe errors
Majority baseline	Real only	33.5%	16.7%	1.000	75
scikit-learn RF	Real only	44.6% $\pm$ 8.5%	44.4% $\pm$ 8.4%	0.750	44
Custom RF	Real only	42.4% $\pm$ 6.7%	41.6% $\pm$ 6.8%	0.777	45
scikit-learn RF	+3,000 synthetic/fold	38.4% $\pm$ 4.8%	30.6% $\pm$ 3.7%	0.902	64
Custom RF	+3,000 synthetic/fold	37.5% $\pm$ 3.7%	30.0% $\pm$ 2.9%	0.920	66

Interpretation

The real-only scikit-learn forest is the best tested model. It improves accuracy by 11.1 percentage points and macro-F1 by 27.7 points over the baseline. Recall remains modest—48.0% for Low, 44.6% for Medium, and 41.3% for High—and 44 predictions are severe Low $\leftrightarrow$ High mistakes (19.6%).

The synthetic-data hypothesis failed. Adding 3,000 generated rows per fold reduced both accuracy and macro-F1; the augmented scikit-learn model predicted no Medium cases. The generated distribution overwhelmed roughly 179 real training rows per fold without preserving the class decision structure. We therefore select the real-only model and report augmentation as a negative result.

Generalization warning: mean training accuracy is 85.0% versus 44.6% held-out accuracy—a 40.5-point gap. The small single-city sample, assessment-value proxy, random rather than temporal validation, and lack of external/prospective testing prevent operational use.

Risk mitigation already used

- Outcome columns excluded from inputs.
- Every test row is real and held out.
- Fixed seeds, baseline comparison and class-sensitive metrics.
- Severe ordered errors reported explicitly.

Recommended next iteration

- Audit matching, label thresholds, distance units and outliers.
- Collect more real data and reserve a later-year/external test set.
- Tune within nested validation and calibrate probabilities.
- Validate synthetic distributions before reuse.

2. Feature table

Completeness follows the rubric's 0–5 scale. Author cells list contributors responsible for substantial repository implementation of the submitted component.

Description	Platform	Complete	Code	Author(s)	Notes
Prepare the final Calgary modeling table	Local	4/5	Python	Finley Thomson	224-row final CSV is complete and validated for model inputs. Raw-to-final preparation involved manual historical steps and is not the canonical submitted run path.
Generate fold-local synthetic training records	Local	4/5	Python	Finley Thomson; Jarrod Hwang	Seeded, integrated and unit-tested. Training-fold isolation prevents test leakage; current distribution is not class-balanced and reduced performance.
Implement custom decision tree and random forest	Local	4/5	Python	Finley Thomson; Jarrod Hwang	Working and integrated with reproducible seeding. Slower than scikit-learn and covered by limited boundary tests.
Perform leakage-safe stratified cross-validation	Local	5/5	Python	Finley Thomson; Jarrod Hwang	Five folds, real held-out evidence, fold-local scaling/generation, per-fold metrics and deterministic configuration.
Compare baseline and scikit-learn RF models	Local	5/5	Python	Jarrod Hwang	Canonical CLI runs both real-only and augmentation experiments and reports accuracy, macro-F1, ordinal error and severe errors.
Export reproducible evaluation evidence	Local	5/5	Python	Jarrod Hwang	Produces JSON/CSV summaries, fold and class metrics, confusion matrices, comparison plot, dataset hash and run manifest.

Completeness scale applied

5 • Complete Working, tested, integrated and clear for the submitted evaluation scope.

4 • Working Integrated with minor coverage, polish or reproducibility limitations.

3 • Partial Demonstrates the concept but needs manual/non-portable steps.

2 or below Prototype, research, or unimplemented; not claimed as a final feature.

Submitted code organization

```
CMPT310_Group14_Code_Submission/
├─ Model/                # evaluator, CV, metrics, custom tree/forest
├─ Data/Code/            # fold-local synthetic generator
├─ Data/CSVs/Sorted/     # final 224-project modeling table
├─ Artifacts/Milestone2/ # reported figures, CSV and JSON evidence
├─ tests/                # deterministic unit tests
├─ requirements*.txt      # compatible and exact-tested dependencies
├─ run_evaluation.sh
├─ run_tests.sh
└─ README.md             # install, run, outputs and troubleshooting
```

Practical software-quality assessment

Functional suitability

Canonical evaluation, baseline, forests, metrics and artifacts work; no live inference UI is claimed.

Reliability

Fixed seeds and held-out evidence support repeatability; the small dataset creates high variance.

Performance

scikit-learn uses parallel trees and runs quickly; the custom implementation is deliberately opt-in.

Maintainability

Evaluation responsibilities are separated and documented; raw preprocessing remains outside the clean run path.

Compatibility

Runs locally on Python 3.11+ from the included final CSV; exact tested versions are supplied.

Security & usability

No credentials or student IDs are included. README commands and explicit warnings reduce user error and financial risk.

3. External tools, libraries, and data

Software used by the submitted run path

Tool	Purpose
Python 3.12.13	Tested runtime and command-line orchestration.
NumPy	Arrays, random sampling, covariance and numeric calculations.
pandas	CSV I/O, validation and tabular feature handling.
scikit-learn	StratifiedKFold, scaling, baseline, RandomForestClassifier and metrics.
Matplotlib	Comparison and confusion-matrix figures.

External packages are used through their documented public APIs. The submitted final path does not copy an external model implementation; the custom tree/forest source is included for inspection.

Data sources

- City of Calgary, **Building Permits**, dataset c2es-76ed.
- City of Calgary, **Historical Property Assessments (Parcel)**, dataset 4ur7-wsgc.
- City of Calgary, **Calgary Transit Stops**, dataset muzh-c9qc.
- City of Calgary, **Parks Sites**, dataset kami-qbfh.
- City of Calgary, **Schools**, dataset fd9t-tdn2.
- Canada Mortgage and Housing Corporation, **Housing Market Information Portal**, Calgary income tables.

Source pages were checked August 4, 2026. The archive includes the derived modeling CSV required for evaluation, not every upstream municipal extract.

Installation and execution

From the extracted CMPT310_Group14_Code_Submission directory:

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python -m pip install -r requirements.txt
python -m Model.EvaluateModels --models dummy sklearn --output-dir Artifacts/FinalRun
python -m unittest discover -s tests -v
```

The exact versions used for final verification are in requirements-tested.txt. The custom forest is optional because it is slower. Each run should use a new output directory to avoid mixing artifacts.

Reproducibility evidence

- Final dataset: 224 rows with no missing model values.
- Class counts: 75 Low, 74 Medium, 75 High.
- Seed, folds, hyperparameters and feature names recorded.
- Dataset SHA-256 written to the generated manifest.
- Five automated tests pass in a fresh extraction.
- Saved and fresh-run metrics agree.
- Count and normalized confusion matrices are included.
- All submitted test evidence is real and out of fold.

Limitations, portability, and responsible use

The clean submission intentionally scopes reproducibility to evaluation from the included final modeling CSV. Historical raw-data preparation depended on manual extracts and local utilities, so it is not represented as a one-command pipeline. Results may not transfer across cities, time periods, project types, market conditions, or assessment policies.

Responsible-use statement: the output must not be used to approve, reject, price, or finance a real development. It is an educational result based on historical public-data proxies and requires expert interpretation.

References

1. City of Calgary Open Data datasets c2es-76ed, 4ur7-wsgc, muzh-c9qc, kami-qbfh and fd9t-tdn2.
2. Canada Mortgage and Housing Corporation, Housing Market Information Portal.
3. scikit-learn documentation: RandomForestClassifier, StratifiedKFold, StandardScaler and model metrics.
4. Group 14 project repository and generated evidence under Artifacts/Milestone2.